

I tested Spout's stock-to-borrow flow on devnet

[Download the complete PDF](#)

I tested the Spout devnet beta on 10 September and again across several sessions on 17 and 18 September, all with the same wallet (`AXJijsKQXEVrJU72P1aXqzguPBDUWaE3ag3tMbEiTQ11`). One order filled properly and the token arrived on chain. Everything after that either stalled or showed the wrong state.

This report leads with the findings and keeps the caveats in one place, so you can read the table first and the methodology later. All funds used were valueless Solana devnet assets.

Findings

#	Severity	Finding	Seen in	Status
1	Blocker	Borrow advertises available capacity, then rejects the same request against a <code>\$0.00</code> limit after a collateral decode error	10 Sep	Confirmed, screenshots and written log
2	Blocker	Orders reach "signed" and are never broadcast. No transaction, no order record	17 and 18 Sep	Confirmed against RPC
3	Blocker	Cancelling an order stalls on the same "sending to the network" message	18 Sep	Confirmed
4	Blocker	A finalized buy order never settled into a holding	17 Sep	Confirmed on chain
5	Blocker	A finalized cancellation returned no refund I could find	17 to 19 Sep	No refund after 22 hours
6	High	A funded wallet displays <code>\$0 USDC</code> because the page's own CSP blocks the devnet RPC	17 and 18 Sep	Confirmed, header and console
7	High	The <code>\$1</code> buy button is a silent no-op, on two different tickers	17 and 18 Sep	Confirmed on 2 assets
8	High	Displayed cost basis does not reconcile with the cash committed	10 Sep	Confirmed, cause unknown
9	Medium	The after-hours confirmation says "You own 0.04 NVDA" before any fill	10 Sep	Confirmed
10	Medium	<code>0% Interest ... Always</code> sits above a table of nonzero annual <code>Borrow Cost</code> values	17 Sep	Confirmed
11	Medium	An order is accepted while the app shows <code>Market: Closed</code> , with no explanation of what happens next	17 Sep	Confirmed
12	Medium	Fee readiness, leverage help and Sell labels need clearer context	10 Sep	Confirmed
13	Docs	Zero withdrawal fee on one page, 0.20% on another	9 Sep	Confirmed

#	Severity	Finding	Seen in	Status
14	Docs	No-lockup language on the homepage, 45-day notice for Junior in the exit policy	9 Sep	Confirmed
15	Docs	Ownership language ignores assignment and liquidation	9 Sep	Confirmed
16	Docs	Junior residual allocation described two different ways	9 Sep	Confirmed
17	Docs	Initial HTML corrupts dollar amounts, the hydrated page is correct	9 Sep	Confirmed

Findings 1 to 12 come from the authenticated beta. Findings 13 to 17 concern published documentation and are marked separately throughout.

Scope and limitations

Worth reading before you weigh the table above.

- **Coverage.** I completed one buy, one fill, and one cancellation. I did not complete a borrow, repayment, sell, Earn deposit, or withdrawal, and nothing here claims otherwise.
- **Why borrowing is untested.** Findings 1 and 2 blocked every borrow attempt before signing, so no debt ever existed to repay or release.
- **Asset coverage.** The deep test on 10 September used NVDA only. The retest added AAPL for findings 2 and 7. No other ticker was transacted.
- **Order broadcast.** Finding 2 rests on traffic visible in the page context plus on-chain checks. If Privy signs inside a separate frame, a broadcast could have escaped that capture. The on-chain side is unambiguous: no new signature ever appeared for this wallet.
- **One contaminated run.** The 17 September retest attempt was affected by my own automation timing. It is treated as supporting evidence only and is not the basis for any finding here.
- **Settlement window.** Finding 5 covers 22 hours, which included a full market session. A longer window remains possible.
- **Not verified by design.** Real-world custody, reserves, KYC, brokerage ownership and investment performance were not independently verified, and nothing was tested on mainnet.
- **Severity labels.** "Blocker" means it stopped my flow in this beta account. It is not a security rating and not evidence of production-wide impact.
- **Docs findings.** Findings 13 to 17 are page-level contradictions captured on 9 September 2026 and may since have been fixed.

The blockers

1. Borrow shows capacity it will not honour

My 10 September NVDA order filled, `0.045302013 NVDA` showed in Portfolio at roughly `$9.88`, and Borrow displayed about `$4.94` available. The wallet still held 10 test USDC plus fee SOL.

I opened Borrow and waited for the data to load. Both the general screen and the NVDA route reported:

```
CollateralType: unexpected length 213 (expected 165, or 149 pre-migration)
```

I selected NVDA, entered 2.40 test USDC, and got a preview at 24% LTV with health factor 4.12. Pressing Borrow failed before any wallet signing with:

```
Borrowing $2.40 exceeds the $0.00 this position supports
```

The screen and the validation were reading different things. One showed positive capacity and a healthy preview, the other resolved the limit to zero. No debt was created, so repayment and collateral release were never reachable.

I would start with the collateral decoder and the schema mismatch the error names. If that read fails, the app should mark collateral unavailable everywhere rather than surfacing a healthy preview and then collapsing to zero at submission. A short recovery message and a diagnostic reference would help support trace it.

A regression test should cover current and older supported schemas, an unexpected account length, missing configuration, and delayed loading. Displayed capacity and the submitted limit should come from the same verified read. These are suggested sponsor tests, not tests I ran against the source.

The length error is a lead, not a diagnosis. It does not by itself prove Token-2022, migration code or the user's asset account is at fault.

[Borrow observation log](#) · [Borrow with no eligible collateral afterwards](#)

2. Orders sign, then nothing is broadcast

This is the one I would fix first, because it silently swallows user intent.

On 18 September I placed a \$10 NVDA order. The confirmation dialog was normal: Buy NVDA , Spend \$10.00 on NVDA at the market price , network devnet , estimated fee <\$0.01 . Pressing Place order produced:

```
Order signed. Submitting it to Solana now. This can take a moment.
```

That message stayed on screen for **152 seconds** with no state change. I repeated the same flow on AAPL and saw the identical message for **90 seconds**. No error appeared in the console either time.

The screen says "signed" and "submitting". Neither happened. Checking the wallet afterwards:

- No new transaction signature. The most recent signature on this wallet was still the faucet claim from 09:10 UTC, hours before the attempts.
- No USDC debit. Balance stayed at 30.
- No order record in Portfolio. Not even a Failed row.

I captured the RPC traffic during a full \$10 NVDA attempt:

Host	Method	Count
api.devnet.solana.com	getTokenAccountBalance	12
api.devnet.solana.com	getAccountInfo	5
api.devnet.solana.com	getMultipleAccounts	3
solana-devnet.rpc.privy.systems	getBalance	2
solana-devnet.rpc.privy.systems	getFeeForMessage	1

No `sendTransaction` and no `getLatestBlockhash` appeared in the page context. The presence of `getFeeForMessage` shows the app built a transaction message and priced it, then stopped. See the broadcast caveat in Scope and limitations.

The app should name the state plainly at each step: awaiting signature, signed, broadcast, confirmed, accepted, settled. If no signature exists yet it should say so instead of leaving the user watching a spinner. Retries also need to be safe so a second click cannot create a duplicate order.

Evidence: [NVDA stuck after signing](#) · [AAPL stuck after signing](#) · [earlier reproduction on 17 September](#) · [Portfolio showing only the older failed order](#)

3. Cancelling an order stalls the same way

On 18 September I tried to cancel the unresolved order from 17 September through Portfolio, Open Orders, `Cancel`.

The confirmation was reasonable: `Cancel order. This withdraws your pending order and returns the reserved funds.` After confirming, the dialog moved to:

```
Cancellation signed. Sending it to the network now.
```

It stayed there for **105 seconds** with no state change. Portfolio still showed the order as `Cancelling...` and the underlying transaction count never moved.

Same failure shape as finding 2, on a different action. The dialog promises reserved funds will be returned, then never reports whether that happened.

Cancellation needs the same explicit states, and the final state should carry the refund transaction signature.

4. A finalized order never settled into a holding

On 17 September a `PlaceBuyOrder` transaction did reach devnet and finalize:

- Signature:
`FdR6B5wscrtjPJeUmAK3gKrpkf9iwn1MtBD4yeydPvufp1mYeMuC21zhVF9ufwMw3zyN5Nx48WFT6SSofS77XiG`
- 10 test USDC moved into a program-associated token account

- The log estimated `0.045622519 NVDA`
- Solana returned `err: null`

Spout later displayed this order as `Failed / not settled`. Holdings stayed empty and the Token-2022 NVDA balance stayed at zero. That state held through 19 September, so it was not a timing artifact.

History showed `0.045571491 @ $219.22`, which does not match the finalized placement log. I could not tell whether that figure was a refreshed quote, an accepted amount, an execution attempt, or a settlement amount.

A finalized order needs a reason code and a receipt that reconciles quote, accepted amount, fill and final settlement. A stated settlement window would tell users when to wait and when to escalate. [Portfolio state after the failure](#)

5. Cancellation finalized, no refund verified

The cancellation transaction was:

- Signature: `27xv24m6ajWpa757Ns5cJBSXtwkmSNwtiTApGZGpJbyod7J8atxKbBPTxjCnywo3Hug3YLGhAp4JB2ETRdRMA9Ko`
- Instruction: `RequestCancelOrder`
- Result: `err: null`
- Immediate token movement: none

I found no separate 10 USDC return transfer. The wallet later reached 30 USDC because I claimed another 20 USDC from a faucet, which was not a refund. By 19 September, roughly 22 hours after the cancellation request and after a full market session, the reserved 10 USDC had still not come back.

This does not prove permanent loss. It means no refund was verifiable in that window, and the app never said one was pending.

Cancellation should show requested, acknowledged, refund pending, and refunded as separate states.

Wrong state shown to the user

6. A funded wallet displays `$0 USDC`

The wallet held 30 devnet USDC on Solana, and the beta header displayed `$0 USDC`. The console showed:

```
Fetch API cannot load https://api.devnet.solana.com/. Refused to connect because it violates the document's Content Security Policy.
```

The deployed `connect-src` policy does not include `api.devnet.solana.com`:

```
'self' https://auth.privy.io https://*.rpc.privy.systems
https://explorer-api.walletconnect.com wss://relay.walletconnect.com
```

wss://relay.walletconnect.org wss://www.walletlink.org
https://*.withpersona.com https://axartrdqynqtfclakxru.supabase.co
wss://axartrdqynqtfclakxru.supabase.co

The RPC endpoint the beta reads balances from is absent from its own allowlist. One page load produced 58 CSP violation entries. When I disabled CSP for this controlled diagnostic only, the correct `$30 USDC` appeared immediately. Navigating to a new page restored the policy and the balance went back to `$0`.

I reproduced this across separate sessions on 17 and 18 September, including one with a fresh console and header capture, so it is not a slow page load.

This matters beyond cosmetics. A funded tester sees an empty wallet and is pushed to request more faucet tokens they do not need.

Spout can fix it by allowing the client RPC endpoint or by reading balances through an allowed same-origin backend. A deployment smoke test with a funded wallet would catch it.

Evidence: [wallet showing zero](#) · [correct balance after the diagnostic](#)

7. The `$1` button is a silent no-op

A `$1` quote renders normally and the button reads `Buy < 0.01 NVDA` with `disabled: false`. A real mouse click produces no dialog, no request, no warning and no error. Entering `$10` opens the expected flow immediately.

I repeated this on AAPL with `Buy < 0.01 AAPL` and got the same silent nothing. So it is not a quirk of one ticker.

I would treat this as an undocumented minimum or a small-order button bug rather than a confirmed protocol defect, but a button that looks enabled and does nothing is worth a message either way.

Evidence: [NVDA `\$1` quote](#) · [NVDA button rendered as enabled](#) · [AAPL `\$1` click with no response](#)

8. The displayed cost basis does not reconcile

The NVDA holding detail showed an average buy price of `$196.44` and roughly a `$0.99` gain at `11.1%`. The observed commitment was 10 test USDC and the delivered amount was `0.045302013` tokens. Committed cash divided by delivered units gives about `$220.74` per unit, while `$196.44` times the delivered quantity gives about `$8.90` of cost basis. At a displayed position value near `$9.88`, those two bases imply different returns.

This does not establish the correct accounting policy or any loss of assets. A fee, premium, adjustment or different basis convention could explain it, but none was stated in the view I saw. The `$220.74` figure is cash committed per unit delivered, not an independently identified exchange execution price.

I would show how committed cash became execution value, delivered quantity, fees or credits, and the cost basis used for P&L. Gross and net figures should be labelled and History and holding detail should use the same calculation. If a value is a placeholder it should not feed live P&L.

Evidence: [Borrow and holding observation log](#) · [Fulfillment receipt](#)

Smaller interface items

9. Ownership language before the fill. With the market closed, the confirmation said `Order confirmed`, `Your purchase has been confirmed`, `You own 0.04 NVDA`, `Bought at $223.77` and `Total paid $10.00`, while scheduling the fill for 09:30 ET. Portfolio showed no holdings and the order sat as `Executing`. History said `Bought NVDA` while its own details said `order placed`.

The later successful fill does not remove the earlier presentation problem. Before a fill exists, ownership and acquisition-price language is premature, and a user could try to borrow against an asset the portfolio does not yet hold. I would use `Order accepted`, `awaiting market open`, `Estimated shares` and `Indicative price`, and keep available cash, committed funds and acquired holdings visibly separate. [After-hours observation](#)

10. 0% Interest ... Always above nonzero Borrow Cost. The beta states zero interest permanently, while the asset table lists nonzero annual `Borrow Cost` values for several equities. The app should explain what Borrow Cost measures and whether and how it relates to APR.

11. Orders accepted while the market is closed. The app allowed an order while displaying `Market: Closed`. The confirmation should say whether the order will be queued, repriced at the next session, handled by an operator, or allowed to expire. I found no evidence that the closed market caused the failed order.

12. Fee readiness, leverage help and Sell labels. The zero-SOL purchase correctly failed before signing and explained a need for about 0.002 SOL, but discovering that only at the action point is avoidable friction, so fee SOL should be visible alongside USDC with a funding path. The captured 2x tooltip described half-price funding, zero interest and upside without mentioning amplified losses or liquidation, so it needs a downside illustration; ignoring costs and liquidation, a 10% decline on a 2x position reduces initial equity by about 20%. The zero-holdings screen correctly disabled Sell, but a separate summary paired the entered sale quantity with a `Stocks owned` label, which should distinguish holdings, amount to sell, expected proceeds and remaining quantity. No sale was executed. [Fee validation](#) · [Leverage help](#) · [Sell screen](#)

What I actually executed

The wallet dialog labelled the cluster **DEVNET TEST FUNDS** and stated the tokens have no value. External funding delivered 20 test USDC from Circle and 1 devnet SOL from Pine Stake. Those receipts establish funding only and are not counted as Spout purchases or loans. [Network guidance](#) · [Circle receipt](#) · [Pine Stake receipt](#)

One Spout buy order committed 10 test USDC at 1x. Placement finalized in slot 495881855 with no error, network fee 5,000 lamports, leaving 10 test USDC and 0.999995 devnet SOL. The modal said it would fill at the next market opening, and the order fulfilled at 13:31:09 UTC, 69 seconds after the displayed 13:30 opening. That was a separate protocol fulfillment, not a second user purchase.

- [Placement transaction](#) · [saved placement receipt](#)
- [Fulfillment transaction](#) · [saved fulfillment and balance verification](#)

The fulfillment log identifies `FulfillBuyOrderFreezeGated` for the same wallet and order. The mint delivered 45,302,013 raw units at nine decimals, so **0.045302013 tokens**, corroborated by a separate finalized Token-2022 balance query. The token account is owned by the test wallet and the original order account was closed after fulfillment. The NVDA identity is corroborated by the loaded beta UI rather than inferred from the public mint address.

The acquired token uses **Token-2022**. Looking only at classic SPL Token accounts finds the remaining 10 USDC but misses the holding. Fulfillment was signed by the protocol operator and the user's fee-SOL balance stayed at 0.999995. These are devnet protocol operations, not evidence of regulated real-world share settlement.

Test group	Actual outcome	Coverage boundary
Access, wallet, network, funding	Legitimate invitation, embedded wallet and both funding receipts verified	No real KYC or live assets
Buy preview and validation	Insufficient USDC and absent fee SOL produced recovery instructions; tiny fractions displayed as nonzero	Rejected attempts did not sign
Buy placement and after-hours wait	One 10 test-USDC order placed and initially pending; misleading ownership wording observed	Pending was never reported as a fill
Fill and holding	Protocol fulfillment finalized, 0.045302013 tokens received and shown as NVDA	Devnet token ownership is not brokerage ownership
Borrow with a holding	About <code>\$4.94</code> available, <code>\$2.40</code> preview at 24% LTV and HF 4.12, decode error, then rejection against <code>\$0.00</code>	No loan signature or debt created
Repayment and release	Unreachable because borrowing was blocked before debt creation	No repayment or release claimed
Earn and cycle simulation	Earn showed coming soon with the wallet still connected, no official assignment or liquidation simulation was available	Not presented as executed tests

The 17 and 18 September retests ran three separate attempts with the same wallet and the same devnet assets. Attempt 1 reached a signed and broadcast buy plus a finalized cancellation, then the order showed `Failed / not settled` and no refund appeared. Attempt 2 was affected by my own automation timing, is treated as supporting evidence only, and is not used to diagnose anything. Attempt 3 reproduced the `$0` balance and the post-signing stall without the blockhash error Attempt 2 had exposed. The 18 September session added the AAPL comparison and the RPC capture in finding 2.

The economic flow

The documented borrower lifecycle is to obtain eligible tokenized equities, lock collateral into an options cycle, borrow stablecoins within a 50% LTV ceiling, and repay. After complete repayment, the documented collateral release occurs at the next cycle close. Holding collateral, enrolling in a cycle, taking a loan, repaying debt and releasing shares are distinct states, and the interface should show them separately.

[Borrowing guide](#)

Participant or component	Contribution	Economic outcome to explain
Borrower	Locked equity collateral, exposure to the options cycle	Stablecoin liquidity, retained stock downside, potentially limited upside and partial liquidation
Option buyer	Premium paid for a contractual stock-purchase right	Right to buy at the strike under the option's terms
Lender	Stablecoin liquidity	Income from the described routing and premium allocation, with tranche-dependent risk
Protocol and reserve	Execution, settlement, a funded loss buffer	Protocol fees, operational responsibilities, limited capacity to absorb losses

A covered call combines stock ownership with the sale of a call on that stock. Premium is received for granting an option right. During the option's life, gains above the strike can be surrendered, while a large decline in the stock can still produce substantial losses. That trade is the reason premium exists. [Options Industry Council: covered calls](#)

Spout describes a systematic strategy based on the historical tendency for implied volatility to exceed subsequent realized volatility. That is a rationale for taking risk, not evidence that every cycle is profitable. A useful performance page would separate gross premium from results after assignment, execution costs, protocol charges, skipped cycles and losses. [Premium rationale](#)

The stated lender model also routes idle stablecoins into a money-market venue, so yield depends on utilization and the underlying venue as well as options outcomes. Target yields should stay clearly labelled as estimates. [Lending model](#)

Two illustrations the interface should provide

Collateral can become unhealthy without interest accruing. Take a position with \$10,000 of collateral, \$5,000 of debt and a hypothetical 60% liquidation threshold. The health factor is $(\$10,000 \times 60\%) / \$5,000 = 1.20$. After a 20% collateral-price decline the collateral is worth \$8,000 and the health factor becomes 0.96. The debt did not grow; the asset supporting it became less valuable. This follows the documented health-factor formula, and the 60% threshold is illustrative, not a claim about any real asset parameter. [Health factor](#)

Before borrowing, the app should show the actual asset threshold, current health factor, liquidation price and a simple decline scenario. A 50% maximum borrowing ratio does not by itself tell a user how far prices can fall before liquidation.

Auto-roll does not necessarily preserve share quantity. Take 100 shares, a \$105 strike, a \$120 expiry price and \$5,000 of outstanding debt. Under the documented sequence, assignment produces \$10,500 of sale proceeds. Repaying debt leaves \$5,500, enough to repurchase roughly 45.83 shares at \$120 before fees or premium allocation. [Assignment sequence](#)

This is not a claim that \$5,000 disappeared, since the loan was repaid. It shows why "position restored" needs a cash-and-stock reconciliation covering extinguished debt, residual cash, repurchased shares and any allocated premium. A new position in the same ticker does not necessarily mean the same number of shares or the same exposure.

Public documentation findings

These concern published information and are not reported as authenticated beta defects.

13. Withdrawal costs need one consistent source. The lending overview presents a zero deposit and withdrawal fee figure, while the dedicated fee page specifies a 0.20% lender withdrawal fee, separate from a potential instant-exit haircut. Those statements lead to different expectations for cash received. Keep one fee rule and show estimated net receipts before an exit request, and do not describe a queued withdrawal as fee-free if another page says otherwise. The current fee page also describes an asset-dependent liquidation penalty, so a previously indexed fixed 5% figure is outdated and excluded from this review. [Lending overview](#) · [Fee schedule](#) · [Zero-fee overview](#) · [0.20% fee table](#)

14. Release timing needs to be stated per position type. The homepage presents broad no-lockup language, while the detailed exit policy specifies at least 45 days' notice for Junior and explains that queued Junior capital stops earning yield while remaining exposed to losses until payment. Senior, Junior and borrower collateral need separate exit summaries showing earliest release date, queue position, estimated payout, exposure while waiting and any fee or haircut. A successful request should not look like completed access to funds. [Homepage](#) · [Exit policy](#) · [Homepage claim](#) · [Junior policy](#)

15. Ownership language should reflect assignment and liquidation. The homepage implies shares are retained in every scenario, while the assignment guide describes sale at strike, debt repayment and optional repurchase from remaining proceeds. The FAQ limits share loss to assignment, although the liquidation page explains collateral can be partially sold during a price decline. Replace absolute ownership language with previews for an unassigned cycle, an assigned cycle and a liquidation, showing stock units and cash movements. The Auto-Roll default and how to change it should appear before collateral is locked. [Homepage](#) · [Assignment](#) · [FAQ](#) · [Liquidation](#)

16. Junior's residual allocation needs consistent wording. The tranche summary describes Junior receiving all remaining yield after Senior's priority, while the worked example and weekly settlement description allocate excess 25% to Senior and 75% to Junior. Publish one settlement formula and use it in the summary, worked example and distribution view. A priority allocation is not a guaranteed return, and if there is a shortfall users should see what was available, which reserve was used and what remained unpaid. [Tranche structure](#) · [Weekly settlement](#) · [Residual wording](#) · [25/75 worked example](#)

17. Initial HTML corrupts monetary examples. Direct retrieval of the liquidation example shows malformed initial markup around dollar amounts. The body contains a root-element fragment followed by `20` where its structured description retains `$120`, and other amounts and metadata are affected the same way. A browser check on 9 September found the intended figures correct after client rendering. This is an initial-HTML and metadata defect, not a claim that the visible beta miscalculates, but search extraction and consumers that do not complete client rendering can receive contradictory amounts. The generated HTML should preserve literal dollar values and metadata should be validated against the hydrated page. I did not establish a root cause from source. [Liquidation example](#)

Risk information that would improve the beta

A visible loss waterfall. The described order is insurance reserve, then Junior capital, then Senior, with an insurance target of 2% of pool value. Users need the current funded balance, the assets backing the

reserve, replenishment rules and drawdown history. A target allocation is not evidence of available capital. [Loss waterfall](#) · [Insurance fund](#)

A clear link to off-chain backing. The docs describe broker-held shares, reserve attestations and token-level transfer restrictions. Asset pages should expose the issuer, mint, custodian, latest attestation and applicable transfer eligibility, so a user can distinguish owning a token, holding a claim through a wrapper, and holding stock in a personal brokerage account. This review did not independently verify those legal or custody relationships. [Security and compliance](#)

Price freshness and market-hours behaviour. The protocol describes oracle safeguards, pauses on stale prices and a different monitoring cadence outside equity-market hours. The borrowing preview should show the price source, its update time and what happens if the underlying market cannot execute a liquidation immediately. That is worth testing with a sponsor-provided simulation rather than inducing a real market failure. [Oracle policy](#)

Risk language consistent with the wrapper. The short borrower-risk page treats the position as adding no failure mode beyond stock ownership, while the general risk disclosure identifies smart-contract, oracle, counterparty, liquidity, stablecoin and regulatory risks. The shorter explanation should point to those exposures at the point of commitment. [Borrower summary](#) · [Risk disclosures](#)

Broker protections should also be described precisely. SIPC does not insure a security against market-price declines, and its existence is not proof that every downstream token-holder claim is covered. [SEC/SIPC guidance](#)

What I would fix first

1. Make the order pipeline report its real state, so signed, broadcast, confirmed, accepted and settled are distinguishable and a stalled order never looks like progress.
2. Give failed orders and cancellations a reason code, a timestamp and a transaction reference, and show refunds as their own tracked state.
3. Fix the CSP rule that blocks the devnet RPC the beta reads balances from.
4. Fix the collateral decoder so displayed capacity and the submitted limit come from the same read.
5. Reconcile quote, accepted quantity, fill, fees, holdings and P&L.
6. Show liquidation and options-cycle scenarios before collateral is locked.
7. Explain Token-2022 controls and the custody boundary on the asset page.
8. Replace the silent `$1` button with an explicit minimum-order message.

Evidence index

- [Spout order-placement receipt](#) and [filled-token receipt with current balances](#) establish the product transactions that did occur.
- [After-hours observation](#) and [post-open Borrow observation](#) are written UI records from the session.
- The 18 September screenshots sit alongside the earlier ones, and the earlier retest captures remain in place: [Earn coming soon](#). [NVDA \\$1](#) quote, [AAPL \\$1](#) no response, [NVDA stalled after signing](#), [AAPL](#)

stalled after signing, Portfolio with only the older failed order.

Final take

I like the direction. Buying a tokenized stock and borrowing against it in one product is genuinely useful when every handoff is visible, and the parts that worked, the buy, the fill and the on-chain token delivery, worked cleanly.

Right now the handoffs are the weak point. Across those sessions I got one completed purchase, a signed order that was never broadcast, a finalized order that never settled, a cancellation with no verifiable refund, and a funded wallet that reported itself empty. Those are the paths I would repair before adding polish.